PBAS System Tools and Technologies

Frontend and Backend Technical Stack with Versions

1. System Architecture

PBAS, Budget Automation System, is built as a web-based system with a separate frontend and backend. The frontend is a React single page application, while the backend is a Ruby on Rails REST API. The frontend communicates with backend APIs under the `/api/v1` namespace.

Layer	Technology	Purpose
Frontend	React with Vite	User interface, forms, dashboards, reports, routing, and API communication.
Backend	Ruby on Rails API	Business logic, authentication, database operations, validations, and reports APIs.
Database	SQLite in current development configuration	Stores users, requests, budgets, payroll, estimates, internal revenue, audit logs, and notifications.
Authentication	JWT and bcrypt	Secure login tokens and password hashing.

2. Frontend Tools and Versions

Tool / Library	Version	Use in System
Node.js	20.20.0	JavaScript runtime used for frontend development and build commands.
npm	10.8.2	Package manager used to install and run frontend dependencies.
React	19.2.0	Main frontend framework for building UI components and pages.
React DOM	19.2.0	Renders React components into the browser DOM.
Vite	7.2.4	Development server and production build tool.
@vitejs/plugin-react	5.1.1	React support inside Vite.
Ant Design	6.0.0	UI components such as forms, buttons, tables, cards, modals, tags, inputs, and layouts.
@ant-design/icons	6.1.0	Icons used in sidebar, buttons, authentication pages, and module actions.
React Router DOM	7.9.6	Frontend routing between dashboard, login, payroll, estimates, reports, settings, and other pages.
Redux Toolkit	2.10.1	State management, especially authentication state and async API actions.
React Redux	9.2.0	Connects React components with the Redux store.
React Easy Crop	5.5.7	Profile image crop functionality in settings/profile area.
ESLint	9.39.1	Frontend code quality and lint checking.
Vitest	4.1.5	Frontend automated testing.
Testing Library React	16.3.2	Testing React components from the user interaction perspective.
jsdom	26.1.0	Browser-like test environment for frontend tests.

Frontend Commands

Command	Purpose
`npm run dev`	Starts the frontend development server.
`npm run build`	Creates production build of the frontend.
`npm run preview`	Previews the production build locally.
`npm run lint`	Runs ESLint to check frontend code quality.
`npm run test`	Runs frontend test suite using Vitest.

3. Backend Tools and Versions

Tool / Gem	Version	Use in System
Ruby	3.3.10	Programming language used for backend development.
Ruby on Rails	8.1.1	Main backend framework for APIs, models, controllers, validations, migrations, and tests.
Puma	7.1.0	Rails application web server.
SQLite	Development database adapter	Current development database configuration uses `storage/development.sqlite3`.
mysql2	0.5.7	MySQL adapter gem available in the backend project.
bcrypt	3.1.20	Secure password hashing for authentication.
jwt	2.10.2	JSON Web Token based authentication/session handling.
rack-cors	3.0.0	Allows frontend and backend to communicate across different local ports/domains.
bootsnap	1.19.0	Improves Rails boot performance.
solid_queue	1.2.4	Rails background job queue support.
solid_cache	1.0.10	Rails cache storage support.
solid_cable	3.0.12	Rails Action Cable storage support.
Brakeman	7.1.1	Static security analysis for Rails code.
RuboCop Rails Omakase	1.1.0	Ruby/Rails code style checking.
Minitest	Rails default test framework	Backend automated tests for APIs and business flows.

Backend Commands

Command	Purpose
`bin/rails server`	Starts the backend Rails server.
`bin/rails db:migrate`	Applies database migrations.
`bin/rails db:seed`	Loads seed/sample data into the database.
`bin/rails test`	Runs backend automated tests.
`bin/rails console`	Opens Rails console to inspect models, records, and database data.
`bin/rails routes`	Shows all backend API routes available in the system.

4. Main System Modules

- **Dashboard:** Shows summary of system data and quick status information.
- **User Management:** Handles users, roles, activation/deactivation, and account management.
- **Resource Request Management:** Handles request creation, verification, approval, rejection, and tracking.
- **Budget Management:** Handles budget heads, categories, allocations, utilization, and remaining budget.
- **Payroll Management:** Stores staff payroll data, allowances, deductions, gross pay, net pay, and active/inactive status.
- **Estimates Management:** Shows estimates, including payroll estimates fetched from payroll data.
- **Internal Revenue:** Stores revenue entries and non-public fund entries with receipt/cutting impact.
- **Reports:** Presents budget, payroll, request, and revenue related reporting data.
- **Audit Logs:** Tracks important system actions for accountability.
- **Settings:** Handles profile, password, image, and account related settings.
- **Notifications:** Shows system notifications and unread/read state.
- **Authentication:** Includes signup, login, email verification, forgot password, and reset password flows.

5. Reports and Charts Technique

The current frontend dependency list does not include a separate chart library such as Recharts or Chart.js. Reports and dashboard visual information are created using React components, Ant Design UI components, tables, cards, progress indicators, filters, CSS styling, and backend API data aggregation.

6. Backend API Coverage

The backend exposes API controllers for sessions, registrations, password reset, email verification, users, workflow requests, budgets, budget heads, budget categories, payrolls, payroll staff, payroll estimates, internal revenue, dashboard, reports, audit logs, notifications, invitations, and profile settings.

> **Summary:** PBAS uses React, Vite, Ant Design, Redux Toolkit, and React Router on the frontend. The backend uses Ruby on Rails API with JWT authentication, bcrypt password hashing, SQLite development database, Rails migrations, REST controllers, and automated tests.